

Digital Nurture 5.0 Deep Skilling Handbook Java FSE - Solution

ALGORITHMS_DATA STRUCTURES

Exercise 1: Inventory Management System

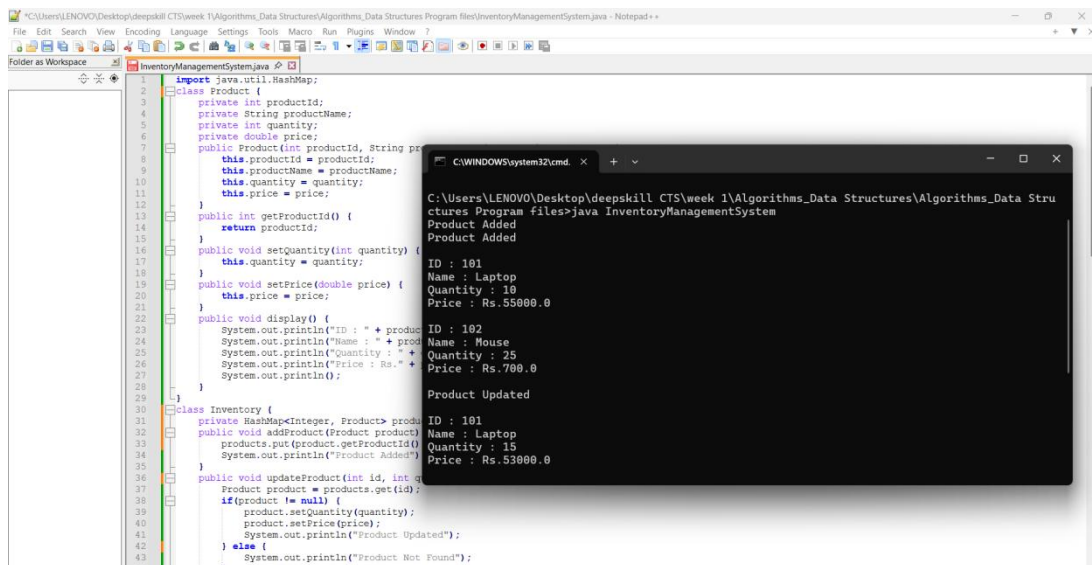
```
import java.util.HashMap;
class Product {
    private int productId;
    private String productName;
    private int quantity;
    private double price;
    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
    public int getProductId() {
        return productId;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public void display() {
        System.out.println("ID : " + productId);
        System.out.println("Name : " + productName);
        System.out.println("Quantity : " + quantity);
        System.out.println("Price : Rs." + price);
        System.out.println();
    }
}
class Inventory {
    private HashMap<Integer, Product> products = new HashMap<>();
    public void addProduct(Product product) {
        products.put(product.getProductId(), product);
        System.out.println("Product Added");
    }
    public void updateProduct(int id, int quantity, double price) {
        Product product = products.get(id);
        if(product != null) {
            product.setQuantity(quantity);
            product.setPrice(price);
            System.out.println("Product Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }
    public void deleteProduct(int id) {
        if(products.remove(id) != null) {
            System.out.println("Product Deleted");
        } else {
            System.out.println("Product Not Found");
        }
    }
    public void displayProducts() {
        for(Product product : products.values()) {
```

Supersetid :8003253

```
        product.display();
    }
}
}
public class InventoryManagementSystem {

    public static void main(String[] args) {
        Inventory inventory = new Inventory();
        Product p1 = new Product(101,"Laptop",10,55000);
        Product p2 = new Product(102,"Mouse",25,700);
        inventory.addProduct(p1);
        inventory.addProduct(p2);
        System.out.println();
        inventory.displayProducts();
        inventory.updateProduct(101,15,53000);
        System.out.println();
        inventory.displayProducts();
        inventory.deleteProduct(102);
        System.out.println();
        inventory.displayProducts();
    }
}
```

Output:



The screenshot shows a Java IDE with the source code of the `InventoryManagementSystem` class and its output. The source code is as follows:

```
import java.util.HashMap;

class Product {
    private int productId;
    private String productName;
    private int quantity;
    private double price;
    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
    public int getProductId() {
        return productId;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public void display() {
        System.out.println("ID : " + productId);
        System.out.println("Name : " + productName);
        System.out.println("Quantity : " + quantity);
        System.out.println("Price : Rs." + price);
    }
}

class Inventory {
    private HashMap<Integer, Product> products;
    public void addProduct(Product product) {
        products.put(product.getProductId(), product);
        System.out.println("Product Added");
    }
    public void updateProduct(int id, int quantity, double price) {
        Product product = products.get(id);
        if (product != null) {
            product.setQuantity(quantity);
            product.setPrice(price);
            System.out.println("Product Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }
}

public class InventoryManagementSystem {
    public static void main(String[] args) {
        Inventory inventory = new Inventory();
        Product p1 = new Product(101, "Laptop", 10, 55000);
        Product p2 = new Product(102, "Mouse", 25, 700);
        inventory.addProduct(p1);
        inventory.addProduct(p2);
        System.out.println();
        inventory.displayProducts();
        inventory.updateProduct(101, 15, 53000);
        System.out.println();
        inventory.displayProducts();
        inventory.deleteProduct(102);
        System.out.println();
        inventory.displayProducts();
    }
}
```

The output of the program is as follows:

```
Product Added
ID : 101
Name : Laptop
Quantity : 10
Price : Rs.55000.0
Product Added
ID : 102
Name : Mouse
Quantity : 25
Price : Rs.700.0
Product Updated
ID : 101
Name : Laptop
Quantity : 15
Price : Rs.53000.0
```

Exercise 2: E-commerce Platform Search Function

```
class Product {

    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
}

public class SearchExample {
    public static void linearSearch(Product[] products, int searchId) {
        for (Product p : products) {
            if (p.productId == searchId) {
                System.out.println("Linear Search");
                System.out.println("Product Found");
                System.out.println("ID      : " + p.productId);
                System.out.println("Name    : " + p.productName);
                System.out.println("Category : " + p.category);
                return;
            }
        }
        System.out.println("Product Not Found");
    }

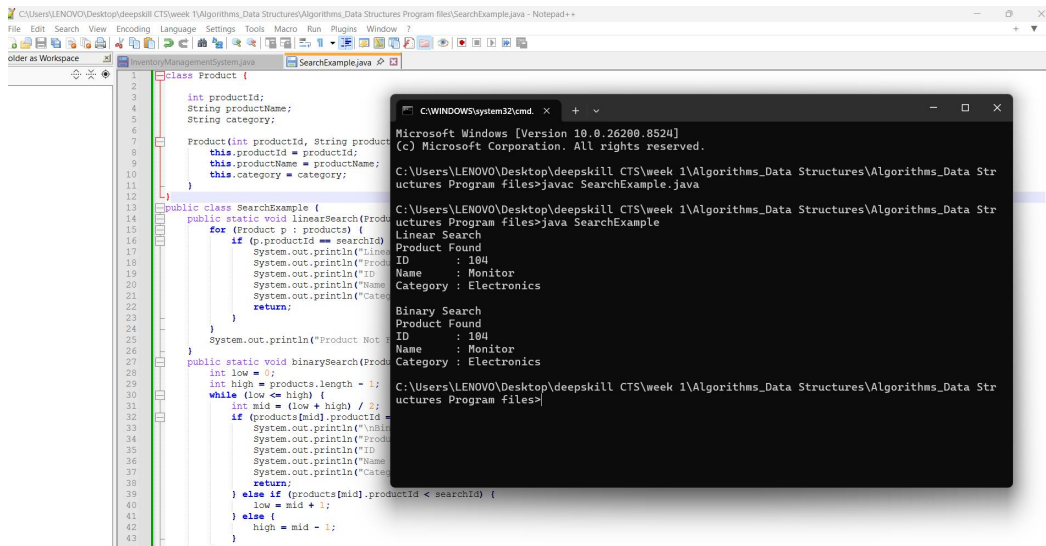
    public static void binarySearch(Product[] products, int searchId) {
        int low = 0;
        int high = products.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (products[mid].productId == searchId) {
                System.out.println("\nBinary Search");
                System.out.println("Product Found");
                System.out.println("ID      : " + products[mid].productId);
                System.out.println("Name    : " + products[mid].productName);
                System.out.println("Category : " + products[mid].category);
                return;
            } else if (products[mid].productId < searchId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        System.out.println("Product Not Found");
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mouse", "Accessories"),
            new Product(103, "Keyboard", "Accessories"),
            new Product(104, "Monitor", "Electronics"),
            new Product(105, "Printer", "Electronics")
        };
    }
}
```

Supersetid :8003253

```
Scanner sc = new Scanner(System.in);
int searchId = sc.nextInt();
linearSearch(products, searchId);
binarySearch(products, searchId);
}
}
```

OUTPUT:



The screenshot shows a Java IDE with the file `SearchExample.java` open. The code defines a `Product` class and a `SearchExample` class. The `SearchExample` class contains two methods: `linearSearch` and `binarySearch`. The `linearSearch` method iterates through a list of products and prints the details of the first product found with the given search ID. The `binarySearch` method uses a binary search algorithm to find the product. The output window shows the results of running the program, displaying the details of the product found for both search methods.

```
1 class Product {
2     int productId;
3     String productName;
4     String category;
5
6     Product(int productId, String productName, String category) {
7         this.productId = productId;
8         this.productName = productName;
9         this.category = category;
10    }
11
12    public class SearchExample {
13        public static void linearSearch(Product[] products, int searchId) {
14            for (Product p : products) {
15                if (p.productId == searchId) {
16                    System.out.println("Linear Search");
17                    System.out.println("Product Found");
18                    System.out.println("ID : 184");
19                    System.out.println("Name : Monitor");
20                    System.out.println("Category : Electronics");
21                    return;
22                }
23            }
24            System.out.println("Product Not Found");
25        }
26
27        public static void binarySearch(Product[] products, int searchId) {
28            int low = 0;
29            int high = products.length - 1;
30            while (low <= high) {
31                int mid = (low + high) / 2;
32                if (products[mid].productId == searchId) {
33                    System.out.println("Binary Search");
34                    System.out.println("Product Found");
35                    System.out.println("ID : 184");
36                    System.out.println("Name : Monitor");
37                    System.out.println("Category : Electronics");
38                    return;
39                } else if (products[mid].productId < searchId) {
40                    low = mid + 1;
41                } else {
42                    high = mid - 1;
43                }
44            }
45        }
46    }
47 }
```

```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>javac SearchExample.java

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>java SearchExample

Linear Search
Product Found
ID : 184
Name : Monitor
Category : Electronics

Binary Search
Product Found
ID : 184
Name : Monitor
Category : Electronics

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>
```

Exercise 3: Sorting Customer Orders

```
class Order {

    int orderId;
    String customerName;
    double totalPrice;
    Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }
    public void display() {
        System.out.println(orderId + " " + customerName + " Rs." + totalPrice);
    }
}

public class SortingExample {
    public static void bubbleSort(Order[] orders) {
        int n = orders.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    public static void quickSort(Order[] orders, int low, int high) {
        if (low < high) {
            int pivot = partition(orders, low, high);
            quickSort(orders, low, pivot - 1);
            quickSort(orders, pivot + 1, high);
        }
    }

    public static int partition(Order[] orders, int low, int high) {
        double pivot = orders[high].totalPrice;
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (orders[j].totalPrice < pivot) {
                i++;
                Order temp = orders[i];
                orders[i] = orders[j];
                orders[j] = temp;
            }
        }
        Order temp = orders[i + 1];
        orders[i + 1] = orders[high];
        orders[high] = temp;
        return i + 1;
    }

    public static void displayOrders(Order[] orders) {
        for (Order order : orders) {
            order.display();
        }
    }
}
```

Supersetid :8003253

```
}  
}  
public static void main(String[] args) {  
    Order[] orders = {  
        new Order(101, "Rahul", 2500),  
        new Order(102, "Anjali", 8000),  
        new Order(103, "Kiran", 1500),  
        new Order(104, "Priya", 6000),  
        new Order(105, "Amit", 4500)  
    };  
    bubbleSort(orders);  
    System.out.println("Bubble Sort:");  
    displayOrders(orders);  
    Order[] orders2 = {  
        new Order(101, "Rahul", 2500),  
        new Order(102, "Anjali", 8000),  
        new Order(103, "Kiran", 1500),  
        new Order(104, "Priya", 6000),  
        new Order(105, "Amit", 4500)  
    };  
  
    quickSort(orders2, 0, orders2.length - 1);  
  
    System.out.println("\nQuick Sort:");  
    displayOrders(orders2);  
}  
}
```

OUTPUT:

The screenshot shows a Java IDE with a source file named `SearchExample.java` and a terminal window displaying the program's output. The source code defines a `Product` class and a `SearchExample` class with methods for linear and binary search. The terminal output shows the results of a linear search for product ID 104, displaying the product details: ID, Name, and Category.

```
class Product {  
    int productId;  
    String productName;  
    String category;  
  
    Product(int productId, String productName, String category) {  
        this.productId = productId;  
        this.productName = productName;  
        this.category = category;  
    }  
  
    public class SearchExample {  
        public static void linearSearch(Product[] products) {  
            for (Product p : products) {  
                if (p.productId == searchId) {  
                    System.out.println("Linear Search  
Product Found  
ID : 104  
Name : Monitor  
Category : Electronics  
");  
                    return;  
                }  
            }  
            System.out.println("Product Not Found");  
        }  
  
        public static void binarySearch(Product[] products) {  
            int low = 0;  
            int high = products.length - 1;  
            while (low <= high) {  
                int mid = (low + high) / 2;  
                if (products[mid].productId == searchId) {  
                    System.out.println("Binary Search  
Product Found  
ID : 104  
Name : Monitor  
Category : Electronics  
");  
                    return;  
                } else if (products[mid].productId < searchId) {  
                    low = mid + 1;  
                } else {  
                    high = mid - 1;  
                }  
            }  
        }  
    }  
}
```

Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>javac SearchExample.java
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>java SearchExample
Linear Search
Product Found
ID : 104
Name : Monitor
Category : Electronics
Binary Search
Product Found
ID : 104
Name : Monitor
Category : Electronics
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>

Exercise 4: Employee Management System

```
class Employee {
    int employeeId;
    String name;
    String position;
    double salary;
    Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }
    public void display() {
        System.out.println(employeeId + " " + name + " " + position + " Rs." + salary);
    }
}

public class EmployeeManagementSystem {
    static Employee[] employees = new Employee[5];
    static int count = 0;
    static void addEmployee(Employee emp) {

        if (count < employees.length) {
            employees[count] = emp;
            count++;
            System.out.println("Employee Added");
        } else {
            System.out.println("Array is Full");
        }
    }

    static void searchEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                System.out.println("Employee Found");
                employees[i].display();
                return;
            }
        }
        System.out.println("Employee Not Found");
    }

    static void displayEmployees() {
        System.out.println("\nEmployee List");
        for (int i = 0; i < count; i++) {
            employees[i].display();
        }
    }

    static void deleteEmployee(int id) {
        for (int i = 0; i < count; i++) {
            if (employees[i].employeeId == id) {
                for (int j = i; j < count - 1; j++) {
                    employees[j] = employees[j + 1];
                }
                employees[count - 1] = null;
                count--;
                System.out.println("Employee Deleted");
                return;
            }
        }
    }
}
```


Supersetid :8003253

```
    }  
}  
System.out.println("Employee Not Found");  
}  
public static void main(String[] args) {  
    addEmployee(new Employee(101, "Rahul", "Manager", 60000));  
    addEmployee(new Employee(102, "Priya", "Developer", 50000));  
    addEmployee(new Employee(103, "Kiran", "Tester", 45000));  
    displayEmployees();  
    System.out.println();  
    searchEmployee(102);  
    System.out.println();  
    deleteEmployee(101);  
    displayEmployees();  
}  
}
```

OUTPUT:

The screenshot displays a Java IDE with two windows. The left window shows the source code for `EmployeeManagementSystem.java`, and the right window shows the output of the program.

Source Code:

```
1 class Employee {  
2     int employeeId;  
3     String name;  
4     String position;  
5     double salary;  
6     Employee(int employeeId, String name, String position, double salary) {  
7         this.employeeId = employeeId;  
8         this.name = name;  
9         this.position = position;  
10        this.salary = salary;  
11    }  
12    public void display() {  
13        System.out.println(employeeId + " " + name + " " + position + " Rs." + salary);  
14    }  
15 }  
16 public class EmployeeManagementSystem {  
17     static Employee[] employees = new Employee[5];  
18     static int count = 0;  
19     static void addEmployee(Employee emp) {  
20         if (count < employees.length) {  
21             employees[count] = emp;  
22             count++;  
23             System.out.println("Employee Added");  
24         } else {  
25             System.out.println("Array is Full");  
26         }  
27     }  
28     static void searchEmployee(int id) {  
29         for (int i = 0; i < count; i++) {  
30             if (employees[i].employeeId == id) {  
31                 System.out.println("Employee Found");  
32                 employees[i].display();  
33                 return;  
34             }  
35         }  
36         System.out.println("Employee Not Found");  
37     }  
38     static void displayEmployees() {  
39         System.out.println("\nEmployee List");  
40         for (int i = 0; i < count; i++) {  
41             employees[i].display();  
42         }  
43     }  
44 }
```

Output:

```
(c) Microsoft Corporation. All rights reserved.  
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>javac EmployeeManagementSystem.java  
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>java EmployeeManagementSystem  
Employee Added  
Employee Added  
Employee Added  
Employee List  
101 Rahul Manager Rs.60000.0  
102 Priya Developer Rs.50000.0  
103 Kiran Tester Rs.45000.0  
Employee Found  
102 Priya Developer Rs.50000.0  
Employee Deleted  
Employee List  
102 Priya Developer Rs.50000.0  
103 Kiran Tester Rs.45000.0  
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>
```

Exercise 5: Task Management System

```
class Task {

    int taskId;
    String taskName;
    String status;

    Task next;
    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}

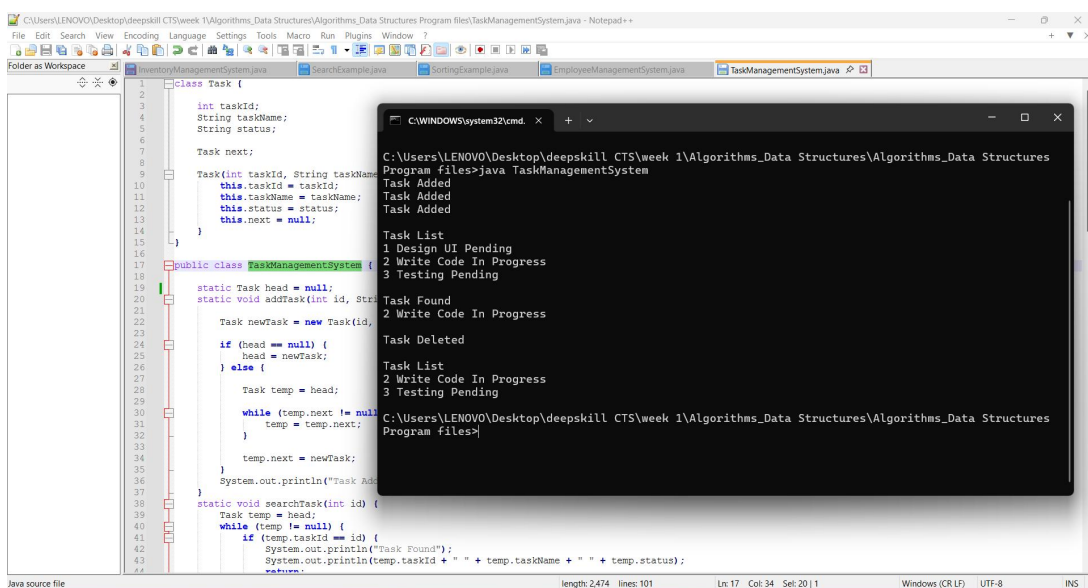
public class TaskManagementSystem {
    static Task head = null;
    static void addTask(int id, String name, String status) {
        Task newTask = new Task(id, name, status);
        if (head == null) {
            head = newTask;
        } else {
            Task temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newTask;
        }
        System.out.println("Task Added");
    }
    static void searchTask(int id) {
        Task temp = head;
        while (temp != null) {
            if (temp.taskId == id) {
                System.out.println("Task Found");
                System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
                return;
            }
            temp = temp.next;
        }
        System.out.println("Task Not Found");
    }
    static void displayTasks() {
        Task temp = head;
        System.out.println("\nTask List");
        while (temp != null) {
            System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
            temp = temp.next;
        }
    }
    static void deleteTask(int id) {
        if (head == null) {
            System.out.println("List is Empty");
            return;
        }
    }
}
```

Supersetid :8003253

```
}
if (head.taskId == id) {
    head = head.next;
    System.out.println("Task Deleted");
    return;
}
Task temp = head;
while (temp.next != null && temp.next.taskId != id) {
    temp = temp.next;
}
if (temp.next == null) {
    System.out.println("Task Not Found");
} else {
    temp.next = temp.next.next;
    System.out.println("Task Deleted");
}
}

public static void main(String[] args) {
    addTask(1, "Design UI", "Pending");
    addTask(2, "Write Code", "In Progress");
    addTask(3, "Testing", "Pending");
    displayTasks();
    System.out.println();
    searchTask(2);
    System.out.println();
    deleteTask(1);
    displayTasks();
}
}
```

OUTPUT:



The screenshot displays a Java IDE with the file `TaskManagementSystem.java` open. The code defines a `Task` class and a `TaskManagementSystem` class. The `main` method performs the following actions: adds three tasks (1: Design UI Pending, 2: Write Code In Progress, 3: Testing Pending), displays the task list, searches for task 2 (found), and deletes task 1. The output window shows the execution results: three 'Task Added' messages, the task list, 'Task Found' for task 2, 'Task Deleted', another task list, and 'Task Deleted' for task 1.

```
class Task {
    int taskId;
    String taskName;
    String status;
    Task next;

    Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}

public class TaskManagementSystem {
    static Task head = null;

    static void addTask(int id, String taskName, String status) {
        Task newTask = new Task(id, taskName, status);
        if (head == null) {
            head = newTask;
        } else {
            Task temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newTask;
        }
        System.out.println("Task Added");
    }

    static void displayTasks() {
        Task temp = head;
        while (temp != null) {
            System.out.println(temp.taskId + " " + temp.taskName + " " + temp.status);
            temp = temp.next;
        }
    }

    static void searchTask(int id) {
        Task temp = head;
        while (temp != null) {
            if (temp.taskId == id) {
                System.out.println("Task Found");
                return;
            }
            temp = temp.next;
        }
        System.out.println("Task Not Found");
    }

    static void deleteTask(int id) {
        if (head.taskId == id) {
            head = head.next;
        } else {
            Task temp = head;
            while (temp.next != null && temp.next.taskId != id) {
                temp = temp.next;
            }
            if (temp.next != null) {
                temp.next = temp.next.next;
            }
        }
        System.out.println("Task Deleted");
    }

    public static void main(String[] args) {
        addTask(1, "Design UI", "Pending");
        addTask(2, "Write Code", "In Progress");
        addTask(3, "Testing", "Pending");
        displayTasks();
        System.out.println();
        searchTask(2);
        System.out.println();
        deleteTask(1);
        displayTasks();
    }
}
```

Output:

```
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures
Program files>java TaskManagementSystem

Task Added
Task Added
Task Added

Task List
1 Design UI Pending
2 Write Code In Progress
3 Testing Pending

Task Found
2 Write Code In Progress

Task Deleted

Task List
2 Write Code In Progress
3 Testing Pending

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures
Program files>
```

Exercise 6: Library Management System

```
class Book {
    int bookId;
    String title;
    String author;
    Book(int id, String title, String author) {
        this.bookId = id;
        this.title = title;
        this.author = author;
    }
}

public class LibraryManagement {
    static void linearSearch(Book[] books, String title) {

        for (int i = 0; i < books.length; i++) {

            if (books[i].title.equalsIgnoreCase(title)) {

                System.out.println("Book Found (Linear Search)");
                System.out.println("Book ID : " + books[i].bookId);
                System.out.println("Title  : " + books[i].title);
                System.out.println("Author : " + books[i].author);
                return;
            }
        }

        System.out.println("Book Not Found");
    }

    static void sortBooks(Book[] books) {
        for (int i = 0; i < books.length - 1; i++) {
            for (int j = 0; j < books.length - i - 1; j++) {
                if (books[j].title.compareToIgnoreCase(books[j + 1].title) > 0) {
                    Book temp = books[j];
                    books[j] = books[j + 1];
                    books[j + 1] = temp;
                }
            }
        }
    }

    static void binarySearch(Book[] books, String title) {
        int low = 0;
        int high = books.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            int result = books[mid].title.compareToIgnoreCase(title);

            if (result == 0) {

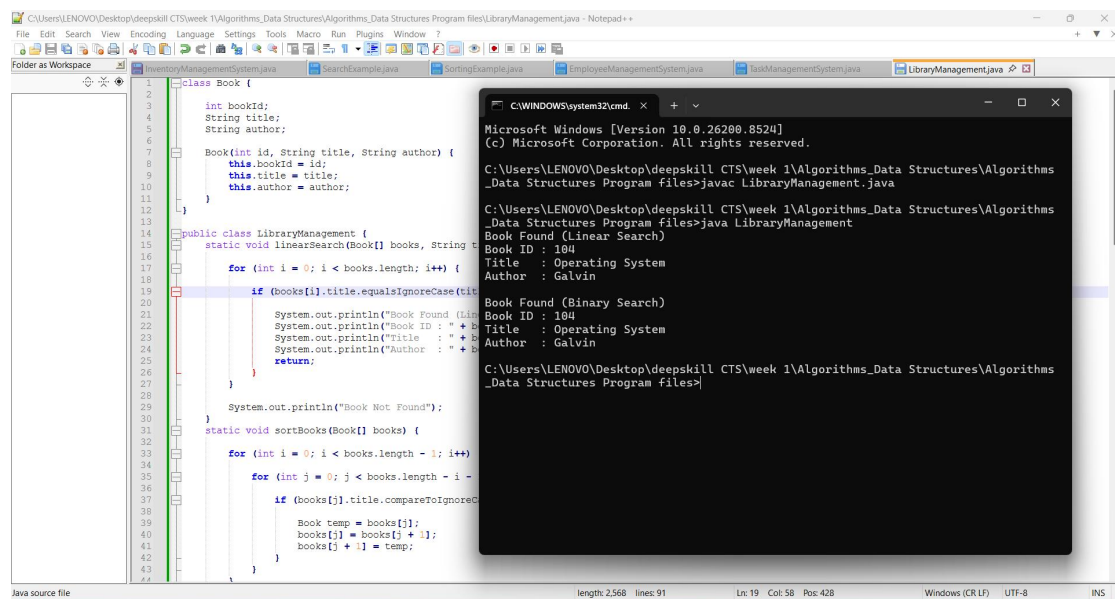
                System.out.println("\nBook Found (Binary Search)");
                System.out.println("Book ID : " + books[mid].bookId);
                System.out.println("Title  : " + books[mid].title);
                System.out.println("Author : " + books[mid].author);
                return;
            } else if (result < 0) {
                low = mid + 1;
            }
        }
    }
}
```

Supersetid :8003253

```
        } else {
            high = mid - 1;
        }
    }
    System.out.println("Book Not Found");
}

public static void main(String[] args) {
    Book[] books = {
        new Book(101, "Java", "James"),
        new Book(102, "Python", "Guido"),
        new Book(103, "C Programming", "Dennis"),
        new Book(104, "Operating System", "Galvin"),
        new Book(105, "Database", "Korth")
    };
    linearSearch(books, "Operating System");
    sortBooks(books);
    binarySearch(books, "Operating System");
}
}
```

OUTPUT:



The screenshot displays a Java IDE with two windows. The left window shows the source code for a library management program. The right window shows the output of the program.

Source Code:

```
1 class Book {
2     int bookId;
3     String title;
4     String author;
5
6     Book(int id, String title, String author) {
7         this.bookId = id;
8         this.title = title;
9         this.author = author;
10    }
11 }
12
13
14 public class LibraryManagement {
15     static void linearSearch(Book[] books, String title) {
16         for (int i = 0; i < books.length; i++) {
17             if (books[i].title.equalsIgnoreCase(title)) {
18                 System.out.println("Book Found (Linear Search)");
19                 System.out.println("Book ID : " + books[i].bookId);
20                 System.out.println("Title : " + books[i].title);
21                 System.out.println("Author : " + books[i].author);
22                 return;
23             }
24         }
25         System.out.println("Book Not Found");
26     }
27
28     static void sortBooks(Book[] books) {
29         for (int i = 0; i < books.length - 1; i++) {
30             for (int j = 0; j < books.length - i - 1; j++) {
31                 if (books[j].title.compareToIgnoreCase(books[j+1].title) > 0) {
32                     Book temp = books[j];
33                     books[j] = books[j+1];
34                     books[j+1] = temp;
35                 }
36             }
37         }
38     }
39 }
40
41
42
43 }
```

Output:

```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms
_Data Structures Program files>javac LibraryManagement.java

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms
_Data Structures Program files>java LibraryManagement
Book Found (Linear Search)
Book ID : 104
Title : Operating System
Author : Galvin

Book Found (Binary Search)
Book ID : 104
Title : Operating System
Author : Galvin

C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms
_Data Structures Program files>
```

Exercise 7: Financial Forecasting

```
public class FinancialForecasting {

    static double futureValue(double amount, double growthRate, int years) {

        if (years == 0) {

            return amount;

        }

        return futureValue(amount * (1 + growthRate), growthRate, years - 1);

    }

    public static void main(String[] args) {

        double presentValue = 10000;

        double growthRate = 0.10;

        int years = 5;

        double result = futureValue(presentValue, growthRate, years);

        System.out.println("Present Value : Rs." + presentValue);

        System.out.println("Growth Rate   : " + (growthRate * 100) + "%");

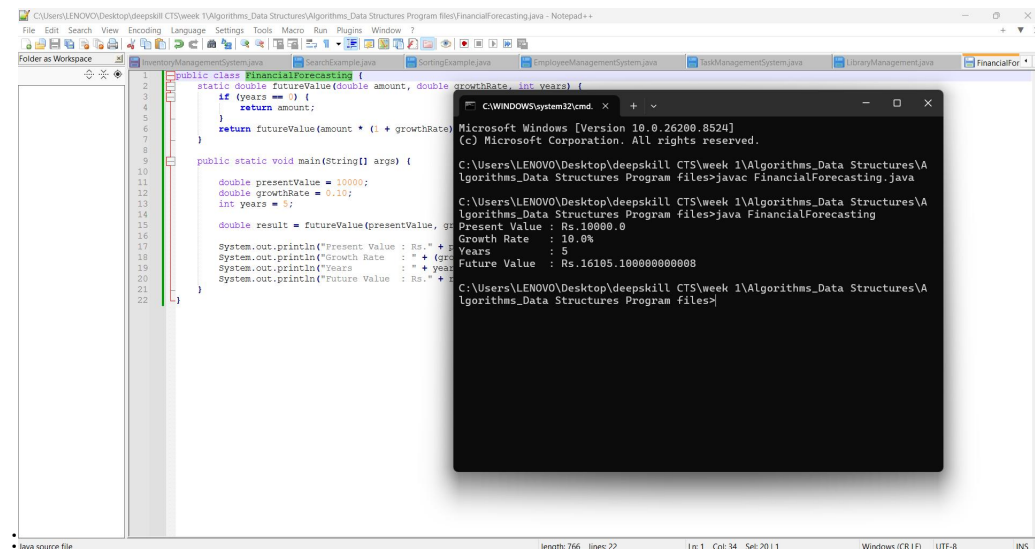
        System.out.println("Years       : " + years);

        System.out.println("Future Value : Rs." + result);

    }

}
```

Output



The screenshot displays a Java IDE with the file `FinancialForecasting.java` open. The code defines a `futureValue` method and a `main` method. The `main` method sets `presentValue` to 10000, `growthRate` to 0.10, and `years` to 5. It then calls `futureValue` and prints the results.

```
1 public class FinancialForecasting {
2     static double futureValue(double amount, double growthRate, int years) {
3         if (years == 0) {
4             return amount;
5         }
6         return futureValue(amount * (1 + growthRate), growthRate, years - 1);
7     }
8
9     public static void main(String[] args) {
10
11         double presentValue = 10000;
12         double growthRate = 0.10;
13         int years = 5;
14
15         double result = futureValue(presentValue, growthRate, years);
16
17         System.out.println("Present Value : Rs." + presentValue);
18         System.out.println("Growth Rate : " + growthRate);
19         System.out.println("Years : " + years);
20         System.out.println("Future Value : Rs." + result);
21     }
22 }
```

The output window shows the following commands and results:

```
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>javac FinancialForecasting.java
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>java FinancialForecasting
Present Value : Rs.10000.0
Growth Rate : 10.0%
Years : 5
Future Value : Rs.16105.100000000008
C:\Users\LENOVO\Desktop\deepskill CTS\week 1\Algorithms_Data Structures\Algorithms_Data Structures Program files>
```